

Prelude

Unlike last term, this homework is **not** optional. Your solution should contain:

`hw3.pdf`: PDF for your answers.

`hw3.erl`: Erlang code if you chose a problem that requires it.

1 The Questions (75 points)

Do any three of the questions below.

1. Tail Recursive Functions (25 points)

- (a) (5 points) Consider our friend, the `count3s` function:

```
count3s([]) -> 0;  
count3s([3 | Tl]) -> 1 + count3s(Tl);  
count3s([_ | Tl]) -> count3s(Tl);
```

Is `count3s` tail-recursive?

Why or why not?

No, by definition a tail-recursive function is one where the recursive call is the last statement that is executed by the function¹. Here each intermediate stack frame must wait to add 1 to the recursive call. Therefore, the recursive call is not the last statement executed by the function.

- (b) (5 points) Write an implementation of `count3s` that is tail-recursive if your answer to Question 1a was “Is not tail-recursive”, and that is not-tail-recursive if your answer to Question 1a was “Is tail-recursive”.

[Tail recursive function implemented in hw3.erl:18-23.](#)

- (c) (3 points) Is the function `reduce/4` in Figure 1 tail-recursive? Give a one-sentence justification for your answer.

No, function calls must wait for the return of recursive calls to perform additional work (ie. message passing).

- (d) (2 points) Is the function `stat/3` in Figure 1 tail-recursive? Give a one-sentence justification for your answer.

Yes, as the final statement executed by the function is a recursive call.

- (e) (3 points) Name two advantages of using tail-recursive functions. At least one of these should be specific to implementing message-passing programs.

Advantage 1 - Memory Efficiency: As tail-recursive functions last executed statement is a recursive call, stack frames can be re-used for the recursive call (tail call elimination). Improving performance by reducing space complexity.

Advantage 2 - Message-Passing Servers: Tail-recursive functions allow message-passing servers to run indefinitely without consuming stack space, as each recursive call to handle the next message reuses the same stack frame.

```

reduce({_}, [], _, GrandTotal) -> GrandTotal;
reduce(ParentPid, [], _, MyTotal) ->
ParentPid ! {self(), reduce_up, MyTotal},
receive
  {ParentPid, reduce_down, GrandTotal} -> GrandTotal
end;
reduce(Parent, [ChildHd | ChildTl], CombineFun, LeftTotal) ->
  receive
    {ChildHd, reduce_up, RightTotal} ->
      GrandTotal =
        reduce(Parent, ChildTl, CombineFun, CombineFun(LeftTotal, RightTotal)),
      ChildHd ! {self(), reduce_down, GrandTotal},
      GrandTotal
  end.
stats(_C0, 0, Acc) -> Acc;
stats(C0, N, Acc) ->
  stats(C0, N-1, stat:accum(island:sim(C0), Acc)).

```

Figure 1: Functions `reduce/4` and `stats/3`.

- (f) **(3 points)** Give a one or two sentence explanation of where it can be a better choice to implement a recursive function without using tail-recursion.
 When intermediate results from each recursive call need to be combined or processed on the return path, non-tail-recursion is preferred because it maintains the call stack to preserve these intermediate values. Examples include tree traversals that aggregate data or mathematical functions that combine results from subproblems.

2. Associative Functions (25 points)

For each function below, state if it is associative or not. If it is associative, give a short proof; if not, give a counter-example. Ignore round-off errors (i.e. all functions are mathematically exact).

If it appears to be associative, but you are stuck with a proof, you can give five test cases to get half credit if it is associative. Your test cases should provide values X , Y , and Z such that

$$f(f(X, Y), Z) == f(X, f(Y, Z)).$$

In fact, if you're stuck with a proof, try a few cases. If you find a counter example you're done. If you don't then you are likely to see *why* the function is associative by looking at the examples you tried.

If you give three test-cases that pass, but the function is not actually associative, you will get 1 point for the examples.

- (a) $a(X, Y) \rightarrow X*Y$. **(5 points)**
- (b) $b(X, Y) \rightarrow 2*X*Y$. **(5 points)**
- (c) $c(X, Y) \rightarrow 2*(X+Y)$. **(5 points)**
- (d) $d(\{X1, X2\}, \{Y1, Y2\}) \rightarrow \{X1+(X2*Y1), X2*Y2\}$. **(5 points)**
- (e) From the four functions listed above, identify one that is associative and commutative. **(2 points)**
- (f) From the four functions listed above, identify one that is associative and not commutative. Given an example for X and Y that shows $f(X, Y) \neq f(Y, X)$, where f is one of the four functions listed above.

3. **Reduce (25 points)** Let $F(X,Y)$ be your choice of $b(X,Y)$, $c(X,Y)$ or $d(X,Y)$ from Question [q:assoc](#), your choice must be a function that you identified as being associative. If you chose b or c , then let $AccIn=0.5$ or $AccIn=0$, respectively, and $List$ be a list of numbers. If you chose d , then let $AccIn=\{0,1\}$ and $List$ be a list of tuples of the form $\{X, Y\}$ where X and Y are numbers. Consider folding a list with your function:

```
Ans = lists:foldl(fun(Element,Acc) -> F(Acc, Element) end, AccIn, List)
```

Write a parallel implementation of this computation:

```
Ans = wtree:reduce(W, LeafFun(ProcState), CombineFun(Left, Right))
```

where W is a worker tree from `wtree:create/1`, and you need to write the functions for `LeafFun` and `CombineFun`.

4. Message Passing Networks

- (a) Bisection Width of a Fat Tree (15 points)

Fat-trees (see slide 20 of the [Sept. 26 lecture slides](#)) are a generalization of tree interconnect that avoid the bandwidth bottleneck at the root by using an increasing number of links between tree nodes when going up in the tree. In particular, if a node is the root of a subtree with M nodes, then there will be $\lceil M^\alpha \rceil$ links from the node to its parent, with $0 \leq \alpha \leq 1$ being a design parameter. Fat trees have been used in real super-computers. For example, the [Tianhe-2 supercomputer](#) held the fastest supercomputer position on the [Top-500 supercomputer list](#) for three years from June 2013 to June 2016 when it was superseded by the Sunway TaihuLight computer. The Tianhe-2 used a fat-tree for its interconnect, see [High Performance Interconnect for Tianhe System](#) – here is the UBC ez-proxy [link](#).

- i. (5 points) What is the bisection width of a Fat Tree with 2^k leaves? Give a formula in terms of k and α .

You can assume that the router-nodes are huge crossbars (see the paper cited above if you want the details of how it can be done in practice, but we won't complicate this question with more detail). You don't need to "look-inside" the router-nodes to answer this question.

$\lceil (2^k - 1)^\alpha \rceil$

(Cutting one root branch based on the formula above, $2L - 1$ nodes in a tree (half of the leaves are present))

- ii. (10 points) Give a short justification of your answer. For example:

- Point out that if the minimum bisection involves cutting one link between two nodes, it must involve cutting all links between those nodes. If you use this in your justification, give a one sentence justification.
- State which links are cut to get the minimum bisection.
- Give a short explanation of why this is the minimum cut. Short means no more than three sentences.

Based on the bisection width of a standard tree "For tree topology with n nodes can be bisected at the root by breaking one link, so bisection bandwidth is one link bandwidth"², the same logic applies to the fat tree. However, in a fat tree with 2^k leaves, this "one link" between the root and a child subtree (containing $2^k - 1$ total nodes) actually consists of $\lceil (2^k - 1)^\alpha \rceil$ parallel links. All of these parallel links must be cut to achieve bisection, because otherwise the two halves would still be connected. Therefore, the bisection width is $\lceil (2^k - 1)^\alpha \rceil$.

- (b) Dimension routing on a torus. (10 points)

Consider a 16×24 torus. Describe for each source-destination pair listed below, give the shortest route, using dimension routing.

- i. **Example:** source = (3,12), destination = (7,6).

Shortest route goes from (3,12) to (7,6) by heading in the direction of increasing x , i.e.

$$(3, 12) \rightarrow (4, 12) \rightarrow (5, 12) \rightarrow (6, 12) \rightarrow (7, 12)$$

using 4 hops. Then it goes from (7,12) to (7,6) by heading in the direction of decreasing y using 6 hops. The total complete route required 10 hops in the network.

Now that I gave one example of the details of “in the direction of increasing x ”, you don’t need to list all the intermediate nodes in your solutions.

- ii. source = (3,12), destination = (10,22).

$$10 - 3 = 7 \text{ hops in increasing } x.$$

$$22 - 12 = 10 \text{ hops in increasing } y.$$

$$7 + 10 = 17 \text{ hops total.}$$

- iii. source = (3,12), destination = (14,12).

$$16 - (14 - 3) = 5 \text{ hops in decreasing } x.$$

$$12 - 12 = 0 \text{ hops in } y.$$

Note: this problem gives more points per byte of answer than the others. Don’t worry about that, just be happy to get an easy question.

5. Communication Overhead (25 points)

Consider the problem of computing the sum of the elements of a list, `List`, of N elements that is distributed across NW workers. Each worker has a segment with N/NW elements of `List`. For simplicity, we will assume that N is a multiple of NW , and that NW is a power of two.

Of course, we will use reduce to compute the sum. The worker tasks will use `lists:sum` to compute their sums. Assume that `lists:sum(L)` takes time $3ns * \text{length}(L)$ where ns is short for “nanosecond”, and one nanosecond is 10^{-9} seconds. Assume that sending one integer between processes takes $2\mu s$ where μs is short for “microsecond”, and one microsecond is 10^{-6} seconds. This is the total time for from when the sending process starts the send until the receiving process completes the receive.

- (a) (4 points) What is the time to compute the sum using reduce?

Note that reduce makes two passes through the tree, one bottom-up and the other top-down whether we use `wtree:reduce` or the book’s method. So, your answer should be the same for either method. We are ignoring compute-time for non-leaf nodes of the tree because it is negligible compared with the communication time.

Leafs are computed in parallel. Communication is done in parallel so it is equal to the height of the tree times 2 (reduce up and reduce down).

$$3ns \cdot (N/NW) + 2(2\mu s \cdot 1000ns / \mu s \cdot \log_2 NW)$$

$$= 3ns \cdot (N/NW) + 4000ns \cdot \log_2 NW$$

- (b) (4 points) What is the time to compute the sum sequentially, i.e. using a single process?

$$3ns \cdot N$$

- (c) (4 points) What is the speed-up?

$$\frac{3 \cdot N}{3 \cdot (N/NW) + 4000 \cdot \log_2 NW}$$

- (d) (3 points) What is the speed-up if $N=100,000$ and $NW=16$?

$$\frac{1200}{139} \approx 8.633$$

- (e) (6 points) Now consider an implementation in C. Assume that C is $10\times$ faster for computing the sum of the elements of an array. In other words, the time to compute the sum of an array with N elements is $0.3ns * N$. Assume that communication costs are the same as the Erlang case (similar costs for any OS overhead, thread synchronization, etc.).

For the questions below, we have given you space to show two or three lines of work for each answer.

- i. What is the parallel time for the C implementation with $N=100,000$ and $NW=16$?
 Leafs are computed in parallel. Communication is done in parallel so it is equal to the height of the tree times 2 (reduce up and reduce down).

$$0.3ns \cdot (N/NW) + 2(2\mu s \cdot \log_2 NW)$$

$$= 0.3ns \cdot \frac{1\mu s}{1000ns} \cdot (N/NW) + 4\mu s \cdot \log_2 NW$$

$$= 17.875\mu s$$
- ii. What is the sequential time for the C implementation with $N=100,000$?

$$100000 \cdot 0.3ns \cdot \frac{1\mu s}{1000ns} = 30\mu s$$
- iii. What is the speed-up for the C implementation with $N=100,000$ and $NW=16$?

$$\frac{0.3 \cdot N}{0.3 \cdot (N/NW) + 4000 \cdot \log_2 NW} = 1.67832$$
- iv. Which is fastest, the Erlang sequential, Erlang parallel, C sequential, or C parallel?
 C parallel.
- v. Which gets the greater speed up, Erlang parallel vs. Erlang sequential, or C parallel vs. C sequential?
 Erlang parallel vs. Erlang sequential.
- vi. Give a one or two sentence explanation for the relationship you observe about which is fastest and which gets the greater speed-up.
 The C parallel version is fastest because even though its speed-up is less than the Erlang implementation, the communication overhead is equal and the parallel summing portions execute faster. The Erlang parallel vs. Erlang sequential experiences greater speed-up because we are dealing with a relatively small problem size ($N=100,000$) where the serializable portion (communication overhead) represents a significant cost, but as you approach infinitely large problem sizes, this serializable portion becomes infinitesimally small compared to the parallel portion, and both speed-ups would approach NW (16 in this case).

References

1. "Tail Recursion - Data Structures and Algorithms," GeeksforGeeks. Available: <https://www.geeksforgeeks.org/dsa/tail-recursion/>
2. "Bisection bandwidth," Wikipedia. Available: https://en.wikipedia.org/wiki/Bisection_bandwidth



Unless otherwise noted or cited, the questions and other material in this homework problem set is copyright 2025 by Susanne Bradley and Mark Greenstreet and are made available under the terms of the Creative Commons Attribution 4.0 International license <http://creativecommons.org/licenses/by/4.0/>